

Modifications par rapport à l'énoncé initial :

- Partie I.D : préciser que la minimalité concerne les automates déterministes complets.
- Questions 26–27 : remplacer $\delta^*(\{I\}, u)$ par $\delta^*(I, u)$, car l'état initial du déterminisé est I .
- Partie II.B.3, avant la question 38 : a est une expression régulière quelconque, pas nécessairement une lettre.
- Justifier les réponses théoriques et les calculs de complexité.
- Lire chaque partie avant de programmer ; les trois parties sont indépendantes.
- « L'indentation, certes utile pour comprendre le code, n'est pas un délimiteur comme en Python. »

I – Mots et automates

I.A – Miroir d'un mot et automate transposé

1. Décrire le langage L_1 de l'automate $\mathcal{A}_1$ de la figure 1 et décrire son langage miroir $\tilde{L}_1$.

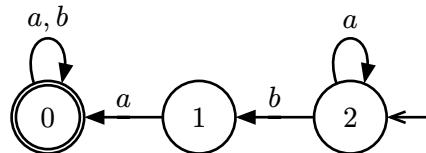
Distinguer un langage d'un ensemble de langages.

Les chemins acceptants bouclent d'abord sur 0, lisent ab pour atteindre 2, puis lisent uniquement des a . Ainsi

$$L_1 = (a \mid b)^* aba^*, \quad \tilde{L}_1 = a^* ba(a \mid b)^*.$$

2. Dessiner un automate $\tilde{\mathcal{A}}_1$ reconnaissant $\tilde{L}_1$.

On inverse toutes les transitions et on échange états initiaux et finaux.



3. Donner, en justifiant, la construction de l'automate miroir $\tilde{A} = (Q, I', F', T')$ reconnaissant $\tilde{L}$.

On prend $I' = F$, $F' = I$ et $T' = \{(q', c, q) \mid (q, c, q') \in T\}$. Un chemin de $i \in I$ à $f \in F$ étiqueté w devient un chemin de f à i étiqueté $\tilde{w}$, et réciproquement. Cela inclut ε : il est accepté dans les deux automates exactement lorsque $I \cap F \neq \emptyset$.

4. Écrire une fonction `transpose` de signature `automate -> automate` qui renvoie un automate reconnaissant le miroir du langage de l'automate d'entrée.

```

let transpose a =
  { nb = a.nb; init = a.final; final = a.init;
    trans = List.map (fun (p, c, q) -> (q, c, p)) a.trans }
  
```

5. Quelle est la complexité de cette fonction ?

Chaque transition est parcourue une fois. Le coût est $O(1 + |T|)$ en temps et en espace supplémentaire ; les listes d'états sont partagées.

I.B – Palindromes et régularité

6. Écrire une fonction `palindrome` de signature `string -> bool` qui teste, en temps linéaire, si un mot est un palindrome.

```

let palindrome s =
  let n = String.length s in
  let rec teste i =
    i >= n / 2 || (s.[i] = s.[n - 1 - i] && teste (i + 1))
  in
  teste 0
  
```

On compare les lettres symétriques jusqu'au milieu du mot. L'appel d'indice i vérifie les paires restantes ; $\lfloor \frac{n}{2} \rfloor - i$ décroît tant qu'un appel récursif est effectué. Le mot vide est accepté. Au plus $\lfloor \frac{n}{2} \rfloor$ comparaisons donnent un coût $O(n+1)$, linéaire dans le pire cas. L'appel récursif est terminal, donc l'espace auxiliaire est constant.

7. Montrer que si Σ est un alphabet à une lettre, alors $\text{Pal}(\Sigma)$ est régulier.

Si $\Sigma = \{a\}$, tout mot est de la forme a^n et est un palindrome. Ainsi $\text{Pal}(\Sigma) = a^*$.

8. Montrer que si Σ contient au moins deux lettres, alors $\text{Pal}(\Sigma)$ n'est pas régulier. On pourra utiliser un automate et un mot de $\text{Pal}(\Sigma) \cap a^*ba^*$.

Choisissons deux lettres distinctes $a, b \in \Sigma$. Si $\text{Pal}(\Sigma)$ était régulier, son intersection K avec a^*ba^* le serait aussi. Or $K = \{a^nba^n \mid n \in \mathbb{N}\}$. Supposons qu'un automate déterministe à p états reconnaisse K . Sur le chemin étiqueté a^pba^p , deux des $p+1$ états atteints après $\varepsilon, a, \dots, a^p$ sont égaux. Il existe donc $0 \leq i < j \leq p$ tels que l'on puisse répéter le segment de $j-i$ lettres a . L'automate accepte alors $a^{p+j-i}ba^p$, qui n'appartient pas à K : contradiction.

9. Montrer que $L_{q,q'}$ est reconnaissable et exprimer L_A en fonction des langages $L_{q,q'}$.

Distiquer une famille de langages de leur union.

L'automate $(Q, \{q\}, \{q'\}, T)$ reconnaît $L_{q,q'}$. Un chemin est acceptant s'il part d'un état initial et arrive à un état final, d'où

$$L_A = \bigcup_{i \in I} \bigcup_{f \in F} L_{i,f}.$$

10. Montrer que $\text{Pal}(\Sigma) \cap (\Sigma^2)^* = \{u\tilde{u} \mid u \in \Sigma^*\}$.

Le langage $(\Sigma^2)^*$ contient exactement les mots de longueur paire. Tout $u\tilde{u}$ est un palindrome de longueur $2|u|$. Réciproquement, si un palindrome a longueur $2n$, sa seconde moitié est le miroir de sa première moitié u . Il vaut donc $u\tilde{u}$. L'égalité reste vraie pour $u = \varepsilon$.

11. Décrire simplement $D(a^*b)$ et $R(a^*b^*a^*)$.

On a

$$D(a^*b) = \{a^n b b a^n \mid n \in \mathbb{N}\}, \quad R(a^*b^*a^*) = a^*b^*.$$

Pour la seconde égalité, si $w\tilde{w} \in a^*b^*a^*$, son préfixe w appartient à ce même langage et n'utilise que a, b . Un tel mot w comportant un b puis un a fait apparaître un facteur ba^+b dans $w\tilde{w}$, impossible dans $a^*b^*a^*$. Ainsi $w \in a^*b^*$; réciproquement $w = a^i b^j$ donne $w\tilde{w} = a^i b^{2j} a^i$.

12. Les langages $D(L)$ et $R(L)$ sont-ils reconnaissables ? On pourra faire intervenir les langages $L_{q,q'}$ définis ci-dessus.

$D(L)$ ne l'est pas toujours : pour $L = a^*b$, le langage $\{a^n b b a^n \mid n \in \mathbb{N}\}$ ne peut être reconnu par un automate fini. Le même argument de répétition d'une boucle dans le premier bloc de a que pour les palindromes donne une contradiction.

En revanche, $R(L)$ est toujours reconnaissable. Un chemin acceptant étiqueté $w\tilde{w}$ passe, après w , par un état $q \in Q$. On a donc

$$R(L) = \bigcup_{i \in I} \bigcup_{q \in Q} \bigcup_{f \in F} (L_{i,q} \cap \widetilde{L_{q,f}}).$$

Chaque langage de chemins est reconnaissable. Le miroir est reconnaissable en inversant les transitions et en échangeant états initiaux et finaux. Les stabilités par intersection et union finies concluent.

I.C – Déterminisation

Pour la représentation binaire, travailler directement sur les entiers : les conversions intermédiaires en listes annulent le gain recherché.

13. Écrire un automate $\mathcal{A}_2$ non déterministe à quatre états reconnaissant $L_2 = (b \mid ab)^*ba$, avec un unique état initial et un unique état final.

```

let a2 = {
  nb = 4; init = [0]; final = [3];
  trans = [(0, 'b', 0); (0, 'a', 1); (1, 'b', 0);
           (0, 'b', 2); (2, 'a', 3)]
}

```

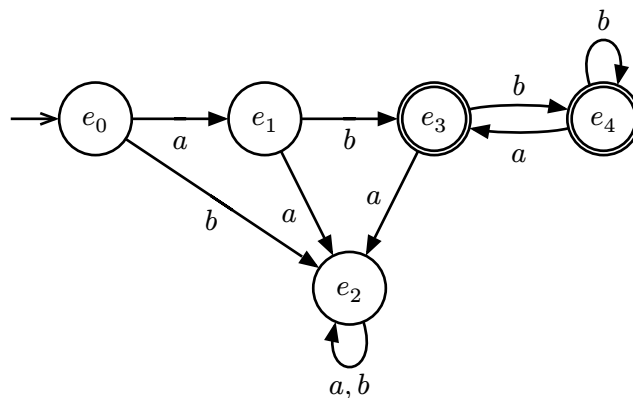
Depuis 0, on peut lire les blocs b ou ab pour revenir à 0. On termine par le chemin 0 vers 2 vers 3, étiqueté ba .

14. Déterminer l'automate miroir $\tilde{\mathcal{A}}_2$ pour obtenir $\mathcal{A}_3 = (\tilde{\mathcal{A}}_2)_{\text{det}}$. Renommer ses états $e_0, e_1, \dots$. Vérifier le déterminisme et les états finaux.

Les états accessibles sont

$$e_0 = \{3\}, \quad e_1 = \{2\}, \quad e_2 = \emptyset, \quad e_3 = \{0\}, \quad e_4 = \{0, 1\}.$$

e_0 est initial ; e_3, e_4 sont finaux.

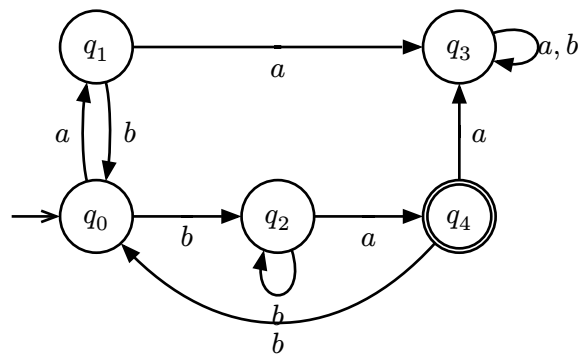


15. Déterminer l'automate miroir $\tilde{\mathcal{A}}_3$ pour obtenir $\mathcal{A}_4 = (\tilde{\mathcal{A}}_3)_{\text{det}}$. Renommer ses états $q_0, q_1, \dots$. Vérifier le déterminisme et les états finaux.

Les états accessibles sont

$$q_0 = \{e_3, e_4\}, \quad q_1 = \{e_4\}, \quad q_2 = \{e_1, e_3, e_4\}, \\ q_3 = \emptyset, \quad q_4 = \{e_0, e_4\}.$$

L'état initial est q_0 ; le seul état final est q_4 .



16. Quel doit être le langage reconnu par $\mathcal{A}_4$?

Chaque détermination préserve le langage, et les deux transpositions prennent deux fois le miroir. Le langage est donc $L_2 = (b \mid ab)^*ba$.

17. Écrire `supprimer : 'a list -> 'a list` qui supprime toutes les occurrences multiples des éléments d'une liste.

```
let rec supprimer = function
| [] -> []
| x :: xs ->
    if List.mem x xs then supprimer xs
    else x :: supprimer xs
```

On conserve la dernière occurrence de chaque élément. L'induction sur la liste montre que le résultat a les mêmes éléments et aucun doublon. Comme avec `List.mem`, l'égalité doit être définie pour les éléments considérés ; ici ce sont des entiers.

18. Donner la complexité en fonction de la taille de la liste.

Pour une liste de longueur m , les recherches parcourent au plus $(m-1) + \dots + 1 = \frac{m(m-1)}{2}$ éléments. Le coût est $O(m^2)$ et cette borne est atteinte quand les éléments sont distincts, en comptant une comparaison en temps constant.

19. Écrire `est_dans : int -> int -> bool` qui teste, à l'aide d'opérations arithmétiques, si $q \in X$ en $O(1)$ opérations.

Le codage binaire permet un test en temps constant.

```
let est_dans q k = (k / pow.(q)) mod 2 = 1
```

Le quotient par 2^q , modulo 2, est le bit d'indice q .

20. Écrire `numero : int list -> int` qui calcule le numéro de X . Par exemple, `[1; 5; 2; 5; 2; 5; 2; 2; 1; 2; 1]` représente $\{1, 2, 5\}$, de numéro $38 = 2^1 + 2^2 + 2^5$.

```
let numero l =
  let rec ajoute k = function
  | [] -> k
  | q :: qs ->
      ajoute (if est_dans q k then k else k + pow.(q)) qs
  in
  ajoute 0 l
```

On ajoute 2^q seulement si le bit correspondant n'est pas déjà présent. Le coût est linéaire dans la longueur de la liste.

21. Écrire `intersecte : int list -> int -> bool` qui vérifie si un élément de ℓ appartient à X .

```
let rec intersecte l k =
  match l with
  | [] -> false
  | q :: qs -> est_dans q k || intersecte qs k
```

22. Écrire `etat_suivant : int -> (int * char * int) list -> int * int` qui, à partir de $k = \text{numero}(X)$ et de T , renvoie $(k_a, k_b) = (\text{numero}(\delta(X, a)), \text{numero}(\delta(X, b)))$.

Éviter les parcours répétés de toutes les transitions et les conversions en listes.

```
let etat_suivant k transitions =
  let ajoute q x = if est_dans q x then x else x + pow.(q) in
  let rec parcours ka kb = function
  | [] -> (ka, kb)
  | (p, c, q) :: ts ->
      if not (est_dans p k) then parcours ka kb ts
      else if c = 'a' then parcours (ajoute q ka) kb ts
      else parcours ka (ajoute q kb) ts
```

```
in
parcours 0 0 transitions
```

Chaque transition issue de X ajoute son arrivée à l'accumulateur de sa lettre. Les doublons ne changent pas le résultat. Coût : $O(1 + |T|)$.

23. Écrire `cherche` : `int -> (int * int) list -> int` qui renvoie le nouveau numéro associé à k , ou -1 si k est absent.

```
let rec cherche k = function
| [] -> -1
| (x, v) :: xs -> if x = k then v else cherche k xs
```

24. Écrire `determinise` : `automate -> automate` qui calcule le déterminisé accessible. Expliquer brièvement la démarche.

```
let determinise a =
  let depart = numero a.init in
  let noms = ref [(depart, 0)] in
  let nombre = ref 1 in
  let attente = ref [depart] in
  let transitions = ref [] in
  let finaux = ref [] in
  let nom k =
    let v = cherche k !noms in
    if v <> -1 then v
    else begin
      let v = !nombre in
      incr nombre;
      noms := (k, v) :: !noms;
      attente := k :: !attente;
      v
    end
  in
  while !attente <> [] do
    match !attente with
    | [] -> ()
    | k :: suite ->
      attente := suite;
      let v = cherche k !noms in
      if intersecte a.final k then finaux := v :: !finaux;
      let ka, kb = etat_suivant k a.trans in
      let va = nom ka in
      let vb = nom kb in
      transitions := (v, 'a', va) :: (v, 'b', vb) :: !transitions
  done;
  { nb = !nombre; init = [0]; final = !finaux; trans = !transitions }
```

La liste `attente` est une pile des parties découvertes et non encore traitées. Chaque partie reçoit son nom lors de sa découverte et n'est empilée qu'une fois. Le traitement ajoute ses deux transitions et marque l'état final exactement si la partie rencontre les états finaux. La partie vide est conservée lorsqu'elle est accessible : elle fournit le puits du déterminisé complet. À la fin, tous les états accessibles ont été traités, puisqu'il n'existe que 2^n parties possibles.

25. Quelle est la complexité de `determinise` en fonction du nombre n d'états de A et du nombre N d'états de A_{det} ?

Chaque partie traitée parcourt T , la liste des états finaux et effectue trois recherches dans une liste de longueur au plus N . Le coût est $O(n + N(|T| + n + N))$. Sur deux lettres, sans transitions répétées, $|T| \leq 2n^2$, donc $O(Nn^2 + N^2 + n)$. L'espace supplémentaire est $O(N)$, en dehors de l'automate d'entrée. Comme $N \leq 2^n$, le temps peut être exponentiel en n .

I.D – Algorithme de Brzozowski

Raisonnement sur les chemins ; distinguer accessibilité et coaccessibilité.

26. Soient $q \in Q$ et $u \in \Sigma^*$. Montrer que si $q \in \delta^*(I, u)$, alors il existe $w \in \Sigma^*$ tel que $uw \in L$.

L'accessibilité de $\tilde{A}$ fournit un chemin de f à q . En le renversant, on obtient dans A un chemin de q à f , étiqueté par un mot w . Comme $q \in \delta^*(I, u)$, un chemin étiqueté u joint un état initial à q ; leur concaténation accepte uw .

27. Montrer la propriété (*) : si $u^{-1}L = v^{-1}L$, alors $\delta^*(I, u) = \delta^*(I, v)$.

Prenons $q \in \delta^*(I, u)$ et un chemin de q à f étiqueté w . Alors $uw \in L$, donc $vw \in L$. Un chemin acceptant pour vw passe, après v , par un état $q' \in \delta^*(I, v)$, puis lit w jusqu'à f . Dans $\tilde{A}$, les deux chemins de f étiquetés $\tilde{w}$ aboutissent à q et q' . Le déterminisme impose $q = q'$. Cela prouve une inclusion ; l'autre s'obtient en échangeant u, v .

28. Pour un automate quelconque A reconnaissant L , on pose $B = (\tilde{A})_{\det}$. En déduire que $(\tilde{B})_{\det}$ reconnaît L et vérifie (*).

B est déterministe, complet et accessible, et reconnaît $\tilde{L}$. L'automate $\tilde{B}$ a un unique état final et son miroir B est déterministe accessible : le résultat précédent s'applique. Son déterminisé C reconnaît L et vérifie (*).

Il est minimal parmi les déterministes complets : si deux mots conduisent au même état d'un autre déterministe complet reconnaissant L , ils ont les mêmes continuations acceptantes, donc le même quotient $u^{-1}L$. Par (*), ils conduisent aussi au même état de C . Choisir un mot d'accès à chaque état de C donne ainsi des états distincts dans l'autre automate. Celui-ci a au moins autant d'états.

29. Écrire `minimal : automate -> automate` appliquant cette construction. On fera abstraction de la taille des automates générés, possiblement problématique.

```
let minimal a = determinise (transpose (determinise (transpose a)))
```

Cette composition suppose, comme demandé, que les représentations intermédiaires sont disponibles. Le code avec `pow` ne fonctionne effectivement que si chaque appel à `determinise` reçoit au plus 20 états. Cette limite peut être dépassée dès la première déterminisation ; il faudrait alors changer la représentation des ensembles pour appliquer la construction sans cette restriction.

II – Expression régulière associée à un automate

II.A – Simplification d'expressions régulières équivalentes

II.A.1 – Parcours d'une expression

30. Écrire `lettre : exprat -> int` qui renvoie le nombre de lettres présentes dans une expression régulière. Par exemple, pour $E = (a^*b) \mid abba(a \mid \varepsilon)^* \mid \emptyset$, la fonction doit renvoyer 7.

```
let rec lettre = function
| Vide | Epsilon -> 0
| Lettre _ -> 1
| Union (e, f) | Concat (e, f) -> lettre e + lettre f
| Etoile e -> lettre e
```

La preuve suit l'induction structurelle : seuls les nœuds `Lettre` contribuent, chacun pour une unité.

31. Écrire `est_vide : exprat -> bool` qui teste si le langage représenté est vide.

```
let rec est_vide = function
| Vide -> true
| Epsilon | Lettre _ | Etoile _ -> false
| Union (e, f) -> est_vide e && est_vide f
| Concat (e, f) -> est_vide e || est_vide f
```

Une union est vide si ses deux termes le sont ; une concaténation est vide si au moins un terme l'est. Une étoile contient toujours ε . Ces règles donnent la correction par induction structurale.

II.A.2 – Règles de simplification

32. Écrire `se : exprat -> exprat` qui simplifie à la racine une expression de type `Etoile` selon ces règles.

```
let se = function
| Etoile Vide | Etoile Epsilon -> Epsilon
| Etoile (Etoile e) -> Etoile e
| e -> e
```

33. Combien d'applications des règles sont nécessaires pour obtenir l'expression a à partir de E_n ?

Il faut $n + 1$ applications : les n concaténations avec $\emptyset$ sont supprimées de l'intérieur vers l'extérieur, puis $a \mid \emptyset$ devient a . Aucune concaténation extérieure n'est simplifiable avant que sa sous-expression droite ne soit devenue $\emptyset$.

34. Écrire `simplifie : exprat -> exprat` qui simplifie une expression selon ces règles.

Simplifier toute l'expression en profondeur.

```
let sc = function
| Concat (Vide, _) | Concat (_, Vide) -> Vide
| Concat (Epsilon, e) | Concat (e, Epsilon) -> e
| e -> e

let rec simplifie = function
| Union (e, f) -> su (Union (simplifie e, simplifie f))
| Concat (e, f) -> sc (Concat (simplifie e, simplifie f))
| Etoile e -> se (Etoile (simplifie e))
| e -> e
```

On simplifie les fils avant la racine. Par induction, les fils sont irréductibles ; la règle appliquée à la racine renvoie alors soit un fils déjà simplifié, soit un constructeur auquel plus aucune règle ne s'applique. Un parcours suffit. Il est linéaire dans le nombre de nœuds de l'arbre syntaxique fourni.

II.B – Matrices d'expressions régulières

II.B.1 – Somme et produit

35. Écrire `somme : mat -> mat -> mat` qui effectue la somme de matrices de même taille $n \times p$. Quelle est sa complexité ?

```
let somme a b =
  let n = Array.length a in
  let p = Array.length a.(0) in
  Array.init n (fun i -> Array.init p (fun j -> Union (a.(i).(j), b.(i).(j))))
```

Chaque coefficient crée une union en temps constant : coût $\Theta(np)$.

36. Écrire `produit : mat -> mat -> mat` qui effectue le produit de deux matrices de tailles compatibles $n \times p$ et $p \times q$. On ne vérifiera pas cette compatibilité. Quelle est sa complexité ?

```

let produit a b =
  let n = Array.length a in
  let p = Array.length b in
  let q = Array.length b.(0) in
  let c = Array.make_matrix n q Vide in
  for i = 0 to n - 1 do
    for j = 0 to q - 1 do
      for k = 0 to p - 1 do
        c.(i).(j) <- Union (c.(i).(j), Concat (a.(i).(k), b.(k).(j)))
      done
    done
  done;
  c

```

Après k itérations de la boucle intérieure, le coefficient calculé représente l'union des k premiers produits. Il y a npq itérations, chacune en temps constant, donc un coût $\Theta(npq)$, initialisation comprise.

II.B.2 – Étoile d'une matrice de taille deux

37. Donner une expression régulière sur $\{a, b, c, d\}$ pour chacun des langages $L_{i,j}$.

$$L_{0,0} = (a \mid bd^*c)^*, \quad L_{1,1} = (d \mid ca^*b)^*,$$

$$L_{0,1} = a^*b(d \mid ca^*b)^*, \quad L_{1,0} = d^*c(a \mid bd^*c)^*.$$

Un chemin de 0 à 0 est une succession de boucles a ou d'excursions bd^*c ; l'argument est symétrique pour 1. Pour aller de 0 à 1, on lit a^*b jusqu'à la première arrivée en 1, puis un chemin de 1 à 1. Même raisonnement pour 1 vers 0.

II.B.3 – Étoile d'une matrice de taille quelconque

Calculer chaque résultat récursif une seule fois avant de compter les appels et de résoudre la récurrence.

38. Évaluer les complexités des sommes et produits. En déduire que le coût $C(n)$ du calcul de l'étoile vérifie $C(n) = 2C(n-1) + O(n^2)$. En déduire la complexité de cet algorithme.

Posons $m = n - 1$. On calcule et conserve D^* et a^* . D^*C coûte $\Theta(m^2)$, puis $B(D^*C)$ coûte $\Theta(m)$. a^*B coûte $\Theta(m)$ et $C(a^*B)$ coûte $\Theta(m^2)$; l'addition à D coûte $\Theta(m^2)$. On calcule ensuite $(D \mid Ca^*B)^*$ une seule fois. Le produit donnant B' coûte $\Theta(m^2)$; celui donnant C' coûte $\Theta(m)$. Les opérations scalaires coûtent $O(1)$; découpage et recollement coûtent $O(n^2)$. Les deux appels sur des matrices de taille $n - 1$ donnent la récurrence. En la déroulant,

$$\frac{C(n)}{2^n} \leq \frac{C(1)}{2} + K \sum_{j=2}^n \frac{j^2}{2^j}.$$

La somme est bornée : le rapport des termes successifs est inférieur à une constante strictement inférieure à 1 à partir d'un certain rang. Ainsi $C(n) = O(2^n)$, et même $\Theta(2^n)$ puisque l'arbre d'appels contient 2^{n-1} feuilles.

39. Évaluer les complexités des sommes et produits. En déduire $C(n) = 4C(\frac{n}{2}) + O(n^3)$, puis la complexité de l'algorithme.

On conserve les quatre étoiles A^* , D^* , A' et D' pour les réutiliser. Il y a un nombre constant de produits de matrices de taille $\frac{n}{2}$, coûtant chacun $\Theta(n^3)$, et de sommes coûtant $\Theta(n^2)$. Le découpage et le recollement coûtent $\Theta(n^2)$. Au niveau k de l'arbre de récursion, le coût hors appels est $O\left(4^k \left(\frac{n}{2^k}\right)^3\right) = O\left(\frac{n^3}{2^k}\right)$. La somme géométrique sur tous les niveaux est $O(n^3)$; les $4^{\log_2 n} = n^2$ feuilles ne changent pas cette borne. Le coût du premier niveau fournit aussi une minoration : $C(n) = \Theta(n^3)$.

40. Comment gérer une taille n quelconque ? Quelle complexité peut-on obtenir pour M^* ?

On peut compléter M par des lignes et colonnes d'expressions vides jusqu'à la puissance de deux p suivante, avec $n \leq p < 2n$. Aucun chemin ne passe par les nouveaux sommets, sauf les chemins vides sur eux-mêmes. Le bloc supérieur gauche de l'étoile est donc M^* , calculé en $O(p^3) = O(n^3)$.

On peut aussi découper directement en tailles $\lfloor \frac{n}{2} \rfloor$ et $\lceil \frac{n}{2} \rceil$, comme dans le code suivant. Au niveau k , il y a au plus 4^k appels sur des tailles au plus $\lceil \frac{n}{2^k} \rceil$, d'où le même majorant géométrique $O(\frac{n^3}{2^k})$ avant le dernier niveau. Les $O(n^2)$ feuilles donnent encore un coût $O(n^3)$.

41. Écrire `etoile : mat -> mat` qui renvoie l'étoile d'une matrice avec l'algorithme récursif le plus adéquat.

```
let rec etoile m =
  let n = Array.length m in
  if n = 1 then [|[|Etoile m.(0).(0)|]|]
  else begin
    let n1 = n / 2 in
    let a, b, c, d = decouper m n1 (n - n1) in
    let sa = etoile a in
    let sd = etoile d in
    let sab = produit sa b in
    let sdc = produit sd c in
    let ap = etoile (somme a (produit b sdc)) in
    let dp = etoile (somme d (produit c sab)) in
    recoller ap (produit sab dp) (produit sdc ap) dp
  end
```

Chaque sous-matrice carrée passée récursivement est strictement plus petite. Les formules de l'énoncé donnent la correction par récurrence sur n . Les quatre résultats récursifs sont calculés une seule fois chacun, comme requis pour le coût cubique.

II.C – Algorithme de Conway

42. Montrer que $L_A = \mathcal{L}([XM_A^*Y]_{0,0})$, où X est une matrice ligne $(x_0, \dots, x_{n-1})$ et Y une matrice colonne de coefficients $y_0, \dots, y_{n-1}$, avec $x_i, y_j \in \{\emptyset, \varepsilon\}$. Préciser les coefficients en fonction de l'automate.

On choisit $x_i = \varepsilon$ si $i \in I$, et $\emptyset$ sinon ; $y_j = \varepsilon$ si $j \in F$, et $\emptyset$ sinon. Le produit représente exactement

$$\bigcup_{i \in I} \bigcup_{j \in F} \mathcal{L}([M_A^*]_{i,j}) = \bigcup_{i \in I} \bigcup_{j \in F} L_{i,j} = L_A.$$

43. Écrire `langage : automate -> exprat` renvoyant une expression régulière du langage de l'automate. Quelle est sa complexité ?

```
let langage a =
  if a.nb = 0 then Vide
  else begin
    let m = Array.make_matrix a.nb a.nb Vide in
    List.iter (fun (i, c, j) ->
      m.(i).(j) <- Union (m.(i).(j), Lettre c)) a.trans;
    let fermeture = etoile m in
    let resultat = ref Vide in
    List.iter (fun i ->
      List.iter (fun j ->
        resultat := Union (!resultat, fermeture.(i).(j))) a.final) a.init;
    !resultat
  end
```

Le calcul final sélectionne directement les coefficients d'indices initial-final, ce qui donne le même langage que XM_A^*Y . La construction de M_A coûte $O(n^2 + |T|)$, l'étoile $O(n^3)$ et l'union finale $O(|I||F|)$. Le coût total est donc $O(1 + n^3 + |T|)$, soit $O(n^3)$ pour $n \geq 1$ sur un alphabet fixé.

Les sous-expressions sont partagées : un constructeur OCaml conserve des références à ses arguments, sans recopier leurs arbres. Cette borne concerne la représentation partagée produite. Le développement, l’affichage ou la simplification naïve de l’arbre entièrement déplié peut coûter beaucoup plus, et n’est pas effectué ici.

III – Automate des dérivées d'Antimirov

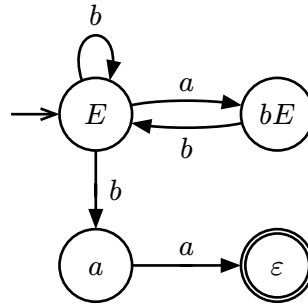
44. Pour $E = (ab \mid b)^*ba$, calculer $\partial_{a(E)}$ et $\partial_{b(E)}$.

Posons $H = (ab \mid b)^*$, donc $E = Hba$. On a $\partial_{a(ab \mid b)} = \{b\}$ et $\partial_{b(ab \mid b)} = \{\varepsilon\}$. Par conséquent $\partial_{a(H)} = \{bH\}$ et $\partial_{b(H)} = \{H\}$. Comme $\varepsilon \in \mathcal{L}(H)$, on obtient

$$\partial_{a(E)} = \{bHba\} = \{bE\}, \quad \partial_{b(E)} = \{Hba, a\} = \{E, a\}.$$

45. Dessiner l’automate obtenu à partir de $E = (ab \mid b)^*ba$. Indiquer précisément son ensemble d’états Q .

On a $Q = \{E, bE, a, \varepsilon\}$, $I = \{E\}$ et $F = \{\varepsilon\}$. En plus des dérivées de E , on a $\partial_{a(bE)} = \emptyset$, $\partial_{b(bE)} = \{E\}$, $\partial_{a(a)} = \{\varepsilon\}$, $\partial_{b(a)} = \emptyset$ et les deux dérivées de ε sont vides. Les quatre états sont accessibles et cet ensemble est stable par dérivation.



L’état final est représenté par un double cercle.

46. Montrer que pour tous mots u, v et tout langage L , $v^{-1}(u^{-1}L) = (uv)^{-1}L$.

Pour tout z , on a successivement

$$z \in v^{-1}(u^{-1}L) \Leftrightarrow vz \in u^{-1}L \Leftrightarrow uvz \in L \Leftrightarrow z \in (uv)^{-1}L.$$

47. Pour tout ensemble S d’expressions sur Σ et tout $w \in \Sigma^*$, montrer $\mathcal{L}(\partial_{w(S)}) = w^{-1}\mathcal{L}(S)$.

Pour une lettre x , l’union des identités admises donne

$$\mathcal{L}(\partial_{x(S)}) = \cup_{E \in S} x^{-1}\mathcal{L}(E) = x^{-1}\mathcal{L}(S).$$

La dernière égalité vient de ce que xz appartient à une union si et seulement s’il appartient à l’un de ses termes. Raisonnons maintenant par récurrence sur la longueur de w , pour tout S . Le cas ε est immédiat car $\partial_{\varepsilon(S)} = S$. Pour $w = ux$, l’identité pour une lettre, l’hypothèse de récurrence et la question 46 donnent

$$\mathcal{L}(\partial_{ux(S)}) = \mathcal{L}(\partial_{x(\partial_{u(S)})}) = x^{-1}(u^{-1}\mathcal{L}(S)) = (ux)^{-1}\mathcal{L}(S).$$

48. Montrer que pour tout $w \in \Sigma^*$, $\partial_{w(E)}$ est l’ensemble des états accessibles depuis E en lisant w .

Pour ε , le seul état atteint est E et $\partial_{\varepsilon(E)} = \{E\}$. Si la propriété est vraie pour u , les états atteints après ua sont les successeurs par a des états de $\partial_{u(E)}$, soit

$$\cup_{H \in \partial_{u(E)}} \partial_{a(H)} = \partial_{a(\partial_{u(E)})} = \partial_{ua(E)}.$$

La récurrence conclut.

49. En déduire que l’automate d’Antimirov reconnaît le langage de E .

Un mot w est accepté si et seulement si un état $H \in \partial_{w(E)}$ est final, c'est-à-dire si $\varepsilon \in \mathcal{L}(\partial_{w(E)})$. Par la question 47, cela équivaut à $\varepsilon \in w^{-1}\mathcal{L}(E)$, donc à $w \in \mathcal{L}(E)$. La finitude de l'automate sera établie à la question suivante.

50. Montrer que $|Q(E)|$ est majoré par le nombre de lettres présentes dans l'écriture syntaxique de E , noté $\|E\|$. Qu'en déduit-on sur l'automate d'Antimirov ?

Raisonnons par induction structurelle.

- Pour $E = \emptyset$ ou $E = \varepsilon$, $Q(E) = \emptyset$ et $\|E\| = 0$.
- Pour une lettre a , $Q(a) = \{\varepsilon\}$ et $\|a\| = 1$.
- D'après (III.1), $Q(E \mid F) = Q(E) \cup Q(F)$, donc $|Q(E \mid F)| \leq |Q(E)| + |Q(F)| \leq \|E\| + \|F\| = \|E \mid F\|$.
- D'après (III.2), $Q(EF) \subseteq Q(E) \cdot \{F\} \cup Q(F)$. L'image de $Q(E)$ par $H \mapsto HF$ contient au plus $|Q(E)|$ éléments. Ainsi $|Q(EF)| \leq \|E\| + \|F\| = \|EF\|$.
- D'après (III.3), $Q(E^*) \subseteq Q(E) \cdot \{E^*\}$, donc $|Q(E^*)| \leq |Q(E)| \leq \|E\| = \|E^*\|$.

L'ensemble des états de l'automate est $\{E\} \cup Q(E)$, car il faut également compter la dérivée par le mot vide. Il contient donc au plus $\|E\| + 1$ états. L'automate est fini, accessible et reconnaît $\mathcal{L}(E)$; il n'est pas nécessairement déterministe.